COMP 3306-P02
Operating Systems
FALL 2026

Group Project
(100 pts)

Overview
Select one of the programming projects from Section 1-3. Analyze what is the problems to be solved, implement the algorithms, test the codes and present in class. Through this project, you can get in-depth knowledge of operating systems and practical programming skills to implement important OS functions.

Topics
Programming questions from basic Operating Systems Concepts

- Chapter 5: CPU Scheduling Simulation
- Chapter 6: Critical Sections & Producer-Consumer Synchronization
- Chapter 9: Dynamic Memory Placement Algorithms Evaluation

Steps & Dates
- Make 4 groups (3 groups of 3 members, 1 group of 4 members), choose a project topic and a group leader
- Please fill out the group information in the project table of eCourses by **October 6**
- Any left-out students by then will be assigned to a random group
- Mid-term presentation **October 27, 2026**
  - Present slides for 5– 7 minutes talk
  - 3/4 presenters each group
    - ✓ Project topic – problem description, assignment to solve
    - ✓ What method will be used – explain main algorithms
    - ✓ What would be the input and output
    - ✓ Timeline & task assignment to each member
    - ✓ Submit a ppt into Canvas,
    - ✓ **October 26**

- Final presentation
  - Final presentation and demonstration for 10-12 minutes talk, **December 1, 2026**

- 3/4 presenters each group
  - ✓ Main algorithms & methods
  - ✓ Implementation & Result (demonstration)
  - ✓ Debug note -- how fixed errors
  - ✓ Conclusion -- What you learned from the project
  - ✓ Role of each member of the group
  - ✓ **Submit a slide ppt, November 30**

- Submission into eCourses
  - Mid-presentation ppt file, **October 27**
  - Final presentation ppt file, **November 30**
  - A compressed zip file which contains source code files, result screen captures, and readme.txt (creation date, authors, overview of codes, etc.) **November 30**

Project Score (100 pts)

- Final zip file – 30 %
- Final presentation – 30 %
- Mid presentation – 30 %
- Peer assessment – 10 %

✓ Presentation evaluation consists of the instructor (60%) and audience (40%)
✓ Group members receive the same grade except peer assessment
✓ Final presentation and zip grades will differ based on peer assessment grade
  - 5% reduced (if peer assessment below 80%)
  - 10% reduced (if peer assessment below 70%)
  - 20% reduced (if peer assessment below 60%)

Project Table

4 groups in total (3 groups of 3 members, 1 group of 4 members)

	Group Name	Members	Leader	Topic
1				
2				
3				
4				

https://docs.google.com/document/d/1ufpnh5DbzdelhzmYDnqmrw4y0zzCcm3o4-Aj5-SqB5c/edit?usp=sharing

1. Dynamic Memory Placement Algorithms Evaluation

Objective
Design and implement a simulator to investigate how First-Fit, Next-Fit, and Best-Fit affect memory usage and execution performance under different workloads.

Project Description
In this project, you will model a fixed-size memory space in which programs continuously request and release memory. Your simulator should keep track of allocated regions and available free regions and update the memory layout as requests are processed.

You may choose an appropriate data structure for representing the memory layout, such as an array combined with linked structures.

Simulation Requirements

1. Create a memory space of size N and initialize it as available memory.
2. Generate memory requests randomly. The requested size should vary around a selected average (for example, 5%, 10%, or 20% of N).
3. Allocate requests using one of the following policies:
    - First-Fit: use the first available region that can satisfy the request.
    - Next-Fit: continue searching from the location of the previous allocation.
    - Best-Fit: select the smallest available region that can satisfy the request.
4. Randomly release previously allocated regions during the simulation.
5. When a region is released, combine it with adjacent free regions whenever possible (coalescing).
6. Continue the allocation and release process for a large number of iterations.
7. Track memory utilization and execution time throughout the experiment.

Tasks

1. **Implementation:** Program the simulator supporting First-Fit, Next-Fit, and Best-Fit routines.
2. **Parametric Evaluation:** Run the simulation across various combinations of total memory size N and mean request sizes.
3. **Performance Analysis:** Benchmark and compare the three allocation strategies, reporting:
    - Average Memory Utilization (%)
    - Total Execution / CPU Time (ms)

2. Critical Sections & Producer-Consumer Synchronization

Objective
To show why thread synchronization is necessary when multiple threads share data, and how to fix race conditions using mutexes and semaphores.

Part 1: Shared Variable and Critical Section

Create a program with two concurrent threads that share an integer variable initialized to 0.

- Thread A repeatedly increases the variable.
- Thread B repeatedly decreases the variable.
- Perform a large number of operations in both threads.

First, run the program without synchronization and record the final value. Repeat the experiment several times and observe whether the result is always consistent.
Then protect the shared update operation using a mutex lock/unlock mechanism. Run the experiment again and verify that the final value is consistent with the expected result.
Briefly explain why the unsynchronized program can produce different results and how the mutex prevents the race condition.

Part 2: Producer-Consumer with a Bounded Buffer

Implement a fixed-size circular buffer shared by two concurrent threads:

- A producer adds items to the buffer.
- A consumer removes items from the buffer.
- Both threads should operate in randomly sized bursts.

First, run the program without synchronization. Demonstrate that the producer may attempt to use a full buffer or the consumer may attempt to remove an item from an empty buffer.
Next, add semaphore-based synchronization so that:

- The producer cannot add an item when the buffer is full.
- The consumer cannot remove an item when the buffer is empty.
- Access to shared buffer information is properly protected.

If your language does not provide the required semaphore operations, implement the necessary P (wait) and V (signal) operations using available mutex/lock primitives.

3. CPU Scheduling Simulation

Objective

Develop a simulation to investigate how different CPU scheduling policies affect the **average turnaround time** of a group of processes.

Implement and compare:

- **FIFO (First-In, First-Out)**
- **SJF (Shortest Job First)**
- **SRT (Shortest Remaining Time)**

Part 1: Scheduling Simulation

Create a set of processes with randomly generated:

- **Arrival time**
- **CPU burst time**
- **Remaining execution time**

At each step of the simulation, determine which processes have arrived and are ready to execute. Select the next process according to the chosen scheduling policy.

For **FIFO**, execute the earliest arriving ready process.

For **SJF**, select the ready process with the shortest CPU burst.

For **SRT**, select the process with the shortest remaining execution time. A newly arriving process may therefore interrupt the currently running process.

Track the following information for each process:

- Arrival time
- CPU execution time
- Completion time
- Waiting time
- Turnaround time

Calculate:

Turnaround Time = Completion Time − Arrival Time

and determine the **average turnaround time** for the entire workload.

Part 2: Experimental Analysis

Repeat the simulation multiple times using randomly generated workloads and calculate the average turnaround time across the runs.

Change the workload parameters, such as:

- Minimum and maximum CPU burst time
- Range of process arrival times
- Number of processes

- Investigate how these changes affect the performance of FIFO, SJF, and SRT.
- Present your results using **tables and/or graphs** and briefly explain the differences you observe among the three scheduling policies.